Assignment 2

Image Classifier using CNN (MNIST)

1. Introduction

The objective of this assignment is to build a Convolutional Neural Network (CNN) model to classify handwritten digits using the MNIST dataset. The model is trained on labelled images of digits (0–9) and the**10. Conclusion**

The CNN model successfully classified handwritten digits with high accuracy on the MNIST dataset. It demonstrated strong performance on test data and reasonable performance on custom images after applying appropriate preprocessing techniques.

n tested on both standard test data and custom handwritten images.

2. Dataset Description

The dataset used is the **MNIST Handwritten Digits Dataset**.

- Total training images: 60,000
- Total testing images: 10,000
- Image size: 28 × 28 pixels
- Color format: Grayscale
- Classes: 10 (digits from 0 to 9)

Each image contains a handwritten digit centered in a 28×28 frame.

3. Data Preprocessing

To prepare the dataset for training, the following preprocessing steps were performed:

3.1 Normalization

Pixel values were scaled from the range 0 - 255 to 0 - 1:

```
x_train = x_train / 255.0
x_test = x_test / 255.0
```

3.2 Reshaping

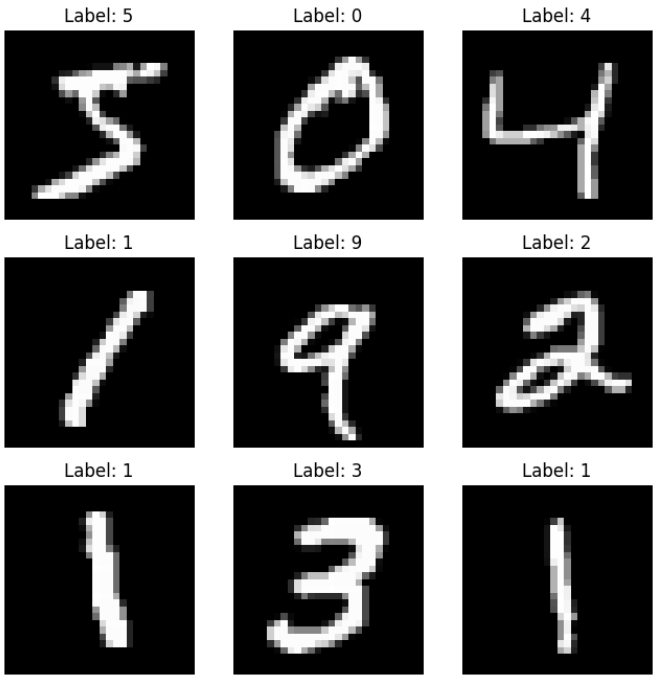
Images were reshaped to include a channel dimension required for CNN input:

```
x_train = x_train.reshape(-1, 28, 28, 1)
```

3.3 One-Hot Encoding

Labels were converted into categorical format:

```
y_train = to_categorical(y_train, 10)
```



MNIST dataset

4. Model Architecture

A Convolutional Neural Network (CNN) was constructed with the following layers:

4.1 Convolutional Layers

These layers extract important features such as edges and patterns from the input images. The ReLU activation function is used to introduce non-linearity.

4.2 Max Pooling Layers

Pooling layers reduce the spatial dimensions of feature maps, which decreases computational complexity and helps control overfitting.

4.3 Dropout Layer

Dropout is used to randomly deactivate neurons during training, which helps prevent overfitting.

4.4 Fully Connected Layer

This layer processes the extracted features and learns complex patterns.

4.5 Output Layer

The output layer consists of 10 neurons corresponding to digits 0–9, using the softmax activation function for multi-class classification.

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)

Trainable params: 225,034 (879.04 KB)

Non-trainable params: 0 (0.00 B)

5. Model Training

The model was trained using the following configuration:

- Optimizer: Adam
- Loss Function: Categorical Crossentropy
- Number of Epochs: 10
- Batch Size: 64
- Validation Split: 20%

During training, the model showed consistent improvement in accuracy while the validation loss decreased, indicating effective learning.

```
Epoch 1/10
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
W0000 00:00:1775636351.695004    3472 cpu_allocator_impl.cc:82] Allocation of 150528000 exceeds 10% of free system memory.
  7/750 ━━━━━━━━━━━━━━━━━━━━ 18s 25ms/step - accuracy: 0.1462 - loss: 2.2931W0000 00:00:1775636352.943988    3621 cpu_allocator_i
exceeds 10% of free system memory.
W0000 00:00:1775636352.945585    3623 cpu_allocator_impl.cc:82] Allocation of 17981568 exceeds 10% of free system memory.
W0000 00:00:1775636352.976576    3623 cpu_allocator_impl.cc:82] Allocation of 17981568 exceeds 10% of free system memory.
W0000 00:00:1775636352.976614    3620 cpu_allocator_impl.cc:82] Allocation of 17981568 exceeds 10% of free system memory.
750/750 ━━━━━━━━━━━━━━━━━━━━ 26s 33ms/step - accuracy: 0.9135 - loss: 0.2809 - val_accuracy: 0.9801 - val_loss: 0.0702
Epoch 2/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 22s 30ms/step - accuracy: 0.9722 - loss: 0.0952 - val_accuracy: 0.9858 - val_loss: 0.0484
Epoch 3/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 41s 30ms/step - accuracy: 0.9789 - loss: 0.0668 - val_accuracy: 0.9882 - val_loss: 0.0420
Epoch 4/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 22s 29ms/step - accuracy: 0.9831 - loss: 0.0565 - val_accuracy: 0.9885 - val_loss: 0.0436
Epoch 5/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 42s 30ms/step - accuracy: 0.9858 - loss: 0.0465 - val_accuracy: 0.9902 - val_loss: 0.0351
Epoch 6/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 22s 30ms/step - accuracy: 0.9875 - loss: 0.0399 - val_accuracy: 0.9898 - val_loss: 0.0354
Epoch 7/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 21s 28ms/step - accuracy: 0.9886 - loss: 0.0361 - val_accuracy: 0.9903 - val_loss: 0.0395
Epoch 8/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 23s 30ms/step - accuracy: 0.9900 - loss: 0.0300 - val_accuracy: 0.9900 - val_loss: 0.0353
Epoch 9/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 22s 30ms/step - accuracy: 0.9915 - loss: 0.0285 - val_accuracy: 0.9911 - val_loss: 0.0336
Epoch 10/10
```

6. Model Evaluation

The trained model was evaluated using the test dataset.

- Test Accuracy Achieved: Approximately 98 - 99%

This demonstrates that the model generalizes well to unseen data and performs accurate classification.

7. Testing with Custom Images

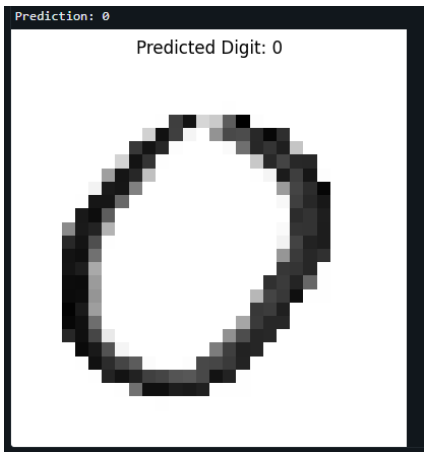
Custom handwritten digit images were used to test the model's real-world performance.

7.1 Preprocessing Steps for Custom Images

- Conversion to grayscale
- Resizing to 28×28 pixels
- Color inversion (white digit on black background)
- Normalization to the range 0–1

7.2 Observations

- Digits such as 0 and 8 were classified correctly



8. Conclusion

The CNN model successfully classified handwritten digits with high accuracy on the MNIST dataset. It demonstrated strong performance on test data and reasonable performance on custom images after applying appropriate preprocessing techniques.